Name: B.Vinayani
Registration Number: 192472179

EXPERIMENT 10

SQL Query Generation using Structured Prompting

AIM

To design structured prompts that provide the database schema, table details and requirements to an LLM and to generate appropriate SQL queries.

PROCEDURE

1. Create a sample SQLite database with two tables, departments and employees, and insert sample records.
2. Design a structured prompt template with sections: Role, Database Engine, Schema, Table Details, Requirement, Rules and Output Format.
3. Define five natural-language requirements (filtering, aggregation with join, top-N, finding unmatched rows and grouping by year).
4. Send each filled prompt to the LLM (the program uses the OpenAI API when OPENAI_API_KEY is set; otherwise stored sample responses are used so that the validation can be demonstrated offline).
5. Extract the SQL from the response, confirm that it is a single SELECT statement and execute it on the database.
6. Compare the number of returned rows with the expected result, display the result tables and capture screenshots.

CODE

Structured prompt template used:

```
### ROLE
You are an expert SQL developer.

### DATABASE ENGINE
SQLite

### SCHEMA
{schema}

### TABLE DETAILS
{details}

### REQUIREMENT
{requirement}

### RULES
- Use only the tables and columns listed in the schema.
- Return a single SELECT statement that ends with a semicolon.
- Use explicit JOIN syntax and meaningful column aliases.

### OUTPUT FORMAT
Return only the SQL inside one ```sql code block.
```

exp10_sql.py

```
"""Experiment 10 - SQL query generation using structured prompting.

The prompt gives the LLM the database engine, schema, table details and a
requirement. The generated SQL is validated (SELECT-only, valid syntax) and then
executed on a sample SQLite database. If OPENAI_API_KEY is not set, stored sample
responses are used so the validation pipeline can still be demonstrated.
"""
import os
import re
import sqlite3

MODEL = "gpt-4o-mini"

SCHEMA = """CREATE TABLE departments (
    dept_id   INTEGER PRIMARY KEY,
    dept_name TEXT NOT NULL,
    location  TEXT
);
CREATE TABLE employees (
    emp_id    INTEGER PRIMARY KEY,
    emp_name  TEXT NOT NULL,
    dept_id   INTEGER REFERENCES departments(dept_id),
    salary    REAL,
    hire_date TEXT   -- format YYYY-MM-DD
);"""

TABLE_DETAILS = """- departments: one row per department (dept_id is the primary
key).
- employees: one row per employee; dept_id is a foreign key to
departments.dept_id."""

PROMPT_TEMPLATE = """### ROLE
You are an expert SQL developer.

### DATABASE ENGINE
SQLite

### SCHEMA
{schema}

### TABLE DETAILS
{details}

### REQUIREMENT
{requirement}

### RULES
- Use only the tables and columns listed in the schema.
- Return a single SELECT statement that ends with a semicolon.
- Use explicit JOIN syntax and meaningful column aliases.

### OUTPUT FORMAT
Return only the SQL inside one ```sql code block."""

SAMPLE_DATA = """
INSERT INTO departments VALUES (1,'Engineering','Chennai'),(2,'HR','Mumbai'),
    (3,'Finance','Delhi'),(4,'Marketing','Bengaluru'),(5,'Legal','Hyderabad');
INSERT INTO employees VALUES
    (101,'Arjun',1,85000,'2021-06-15'),(102,'Meera',1,72000,'2022-01-10'),
    (103,'Kavya',2,48000,'2020-03-01'),(104,'Rahul',3,65000,'2019-11-20'),
    (105,'Sneha',3,52000,'2022-07-05'),(106,'Vikram',1,95000,'2018-09-12'),
    (107,'Divya',4,45000,'2023-02-18'),(108,'Karthik',2,55000,'2021-08-30');
"""
```

```
REQUIREMENTS = [
    {"text": "List the name and salary of all employees earning more than 50000,
highest salary first.",
     "rows": 6,
     "sql": "SELECT emp_name, salary FROM employees WHERE salary > 50000 ORDER BY
salary DESC;"},
    {"text": "Show the average salary of each department along with the
department name, rounded to 2 decimals.",
     "rows": 4,
     "sql": "SELECT d.dept_name, ROUND(AVG(e.salary), 2) AS avg_salary\n"
            "FROM departments d JOIN employees e ON d.dept_id = e.dept_id\n"
            "GROUP BY d.dept_name ORDER BY avg_salary DESC;"},
    {"text": "Find the top 3 highest-paid employees along with their department
names.",
     "rows": 3,
     "sql": "SELECT e.emp_name, d.dept_name, e.salary\n"
            "FROM employees e JOIN departments d ON e.dept_id = d.dept_id\n"
            "ORDER BY e.salary DESC LIMIT 3;"},
    {"text": "List the departments that currently have no employees.",
     "rows": 1,
     "sql": "SELECT d.dept_name FROM departments d\n"
            "LEFT JOIN employees e ON d.dept_id = e.dept_id\nWHERE e.emp_id IS
NULL;"},
    {"text": "Count how many employees were hired in each year.",
     "rows": 6,
     "sql": "SELECT strftime('%Y', hire_date) AS hire_year, COUNT(*) AS
employee_count\n"
            "FROM employees GROUP BY hire_year ORDER BY hire_year;"},
]


def call_llm(prompt, offline_sql):
    key = os.getenv("OPENAI_API_KEY")
    if key:
        from openai import OpenAI
        client = OpenAI(api_key=key)
        reply = client.chat.completions.create(
            model=MODEL, temperature=0,
            messages=[{"role": "user", "content": prompt}])
        return reply.choices[0].message.content, f"OpenAI {MODEL}"
    return f"```sql\n{offline_sql}\n```", "stored sample response (offline mode)"


def extract_sql(text):
    match = re.search(r"```(?:sql)?\s*\n(.*?)```", text, re.DOTALL)
    return (match.group(1) if match else text).strip()


def run_query(conn, sql):
    """Validate and run the SQL. Returns (ok, message, columns, rows)."""
    statements = [s for s in sql.split(";") if s.strip()]
    if len(statements) != 1 or not sql.lstrip().upper().startswith("SELECT"):
        return False, "Only a single SELECT statement is allowed", [], []
    try:
        cursor = conn.execute(sql)
        columns = [col[0] for col in cursor.description]
        return True, "Query is valid and executed", columns, cursor.fetchall()
    except sqlite3.Error as err:
        return False, f"SQL error: {err}", [], []


def print_table(columns, rows):
    widths = [max(len(str(v)) for v in [c] + [r[i] for r in rows])
              for i, c in enumerate(columns)]
    line = "+-" + "-+-".join("-" * w for w in widths) + "-+"
    print(line)
```

```
    print("| " + " | ".join(c.ljust(w) for c, w in zip(columns, widths)) + " |")
    print(line)
    for row in rows:
        print("| " + " | ".join(str(v).ljust(w) for v, w in zip(row, widths)) + "
|")
    print(line)


def main():
    conn = sqlite3.connect(":memory:")
    conn.executescript(SCHEMA)
    conn.executescript(SAMPLE_DATA)

    passed = 0
    for number, req in enumerate(REQUIREMENTS, start=1):
        prompt = PROMPT_TEMPLATE.format(schema=SCHEMA, details=TABLE_DETAILS,
                                        requirement=req["text"])
        if number == 1:
            print("=== Structured prompt sent to the LLM (Requirement 1) ===")
            print(prompt)
        response, source = call_llm(prompt, req["sql"])
        sql = extract_sql(response)
        print(f"\n=== Requirement {number}: {req['text']}")
        print(f"source: {source}\n{sql}")
        ok, message, columns, rows = run_query(conn, sql)
        if ok and len(rows) == req["rows"]:
            passed += 1
            print(f"  [PASS] {message}; {len(rows)} row(s) returned as expected")
            print_table(columns, rows)
        elif ok:
            print(f"  [FAIL] expected {req['rows']} row(s) but got {len(rows)}")
        else:
            print(f"  [FAIL] {message}")

    print(f"\n=== Summary: {passed}/{len(REQUIREMENTS)} generated queries
validated ===")


if __name__ == "__main__":
    main()
```

OUTPUT

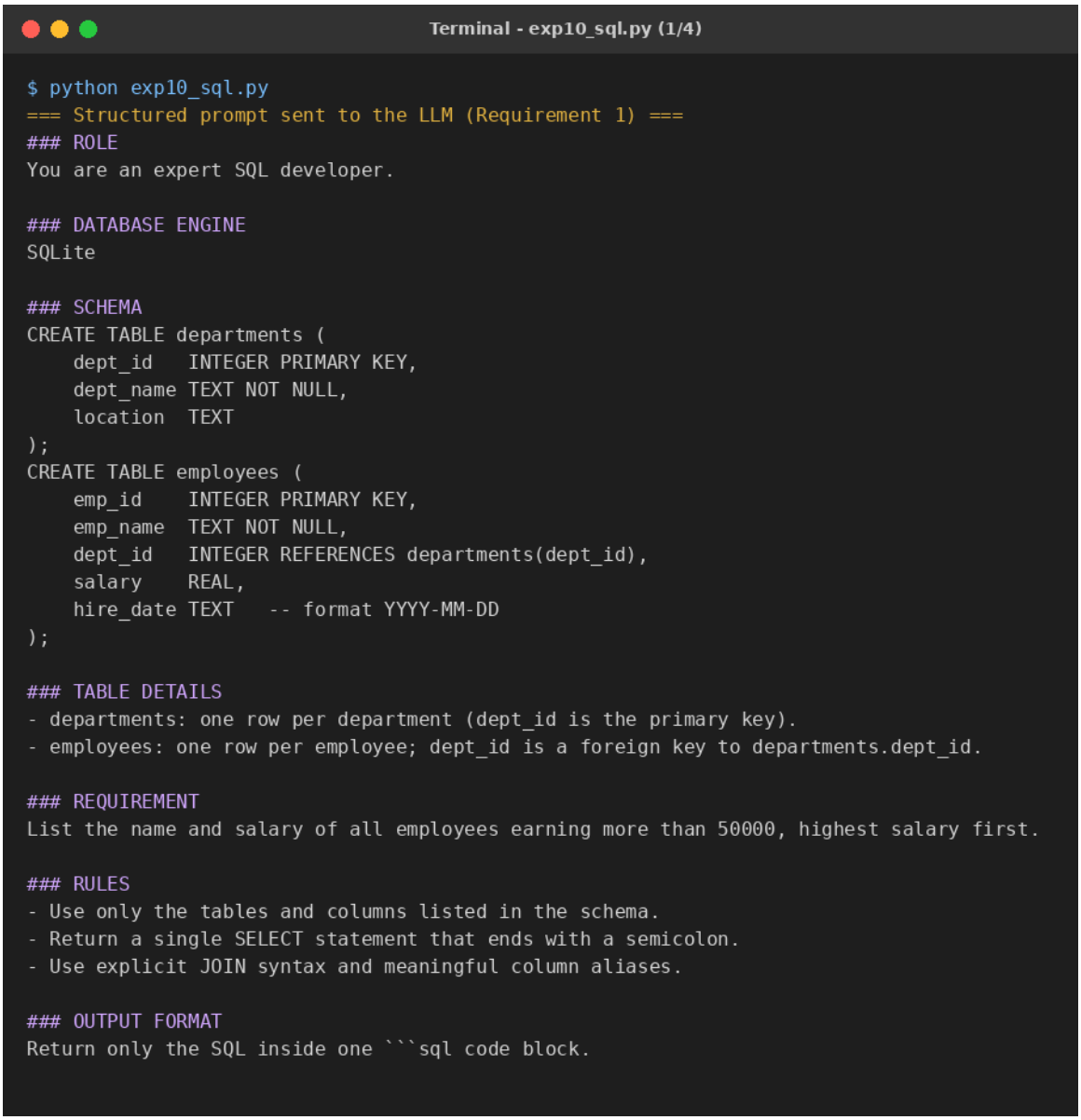


Figure 1: Sample output of the program (part 1 of 4)

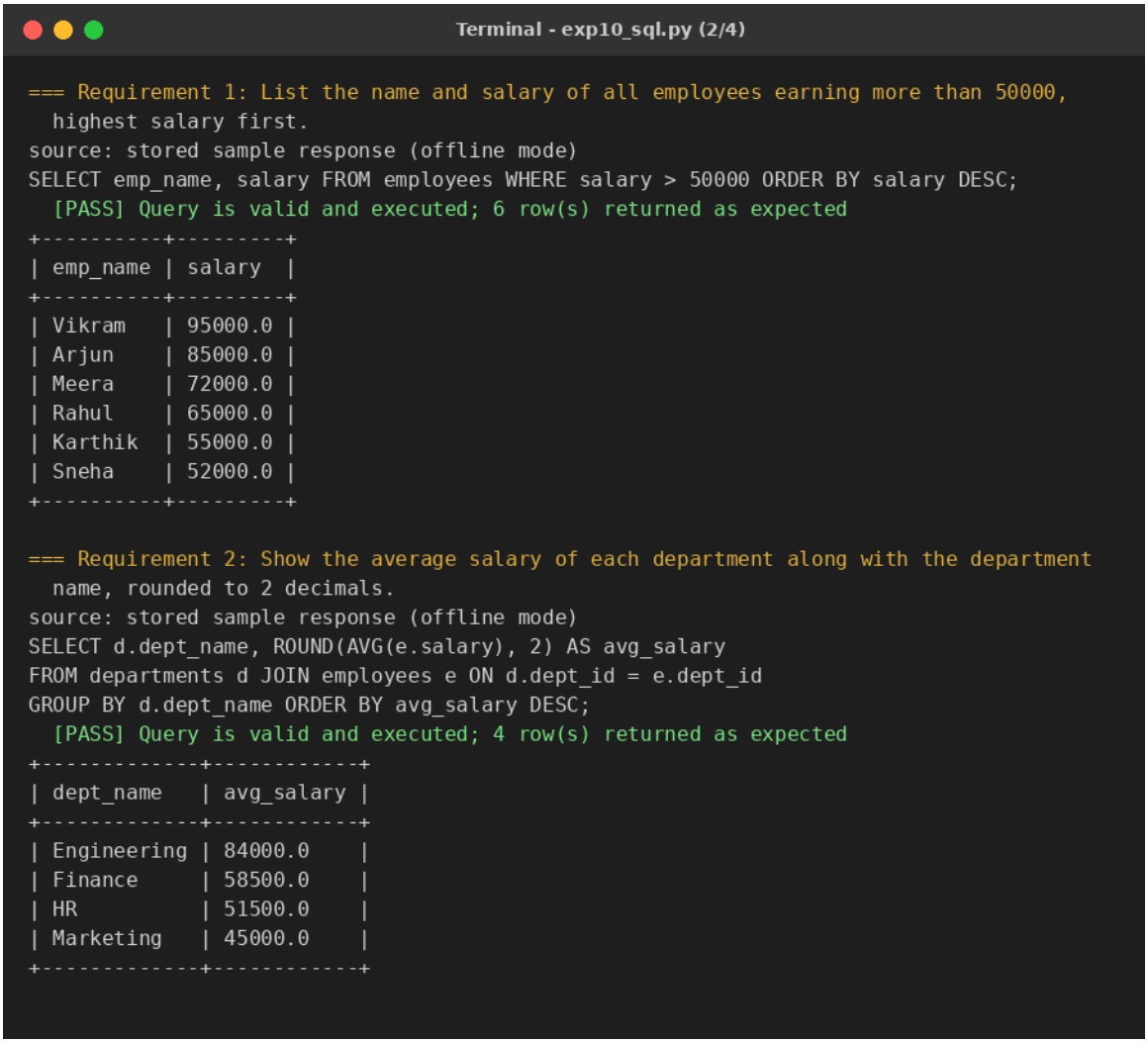


Figure 2: Sample output of the program (part 2 of 4)

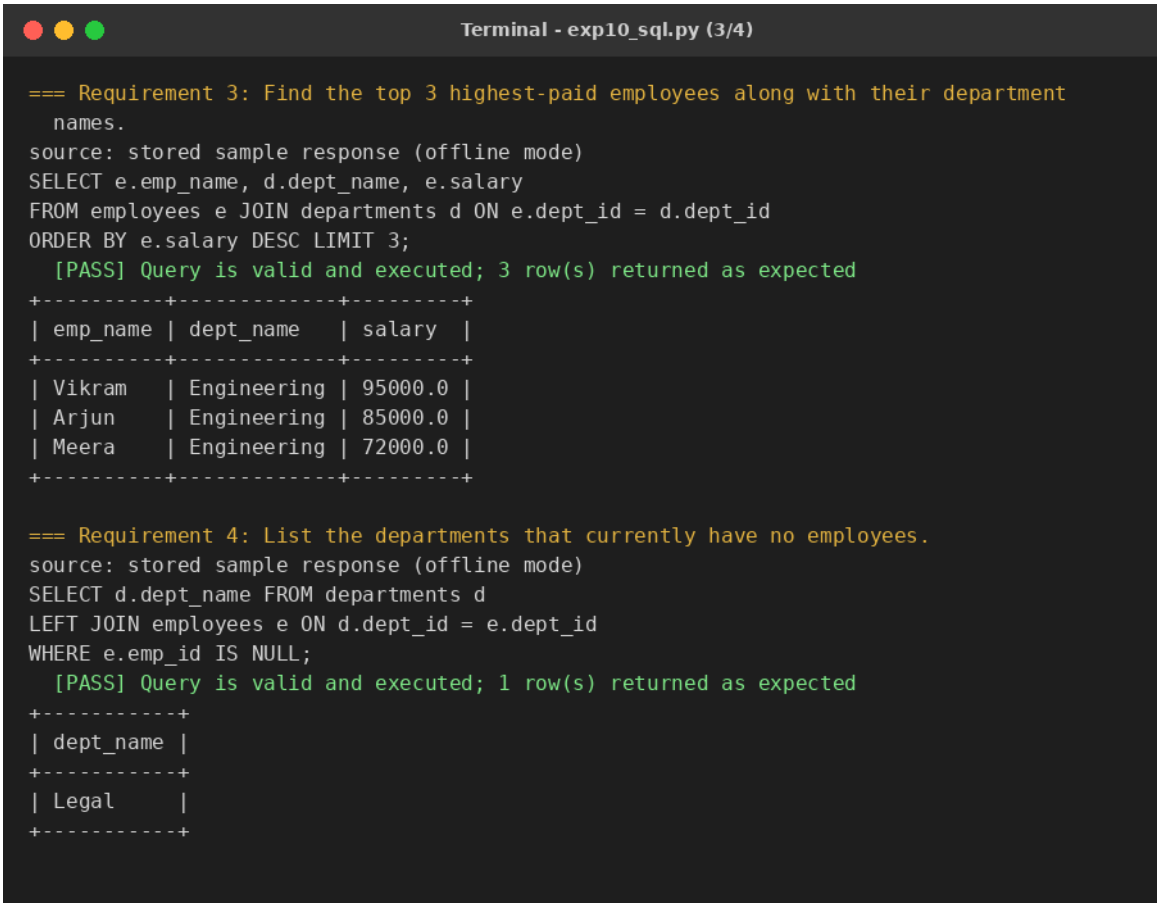


Figure 3: Sample output of the program (part 3 of 4)

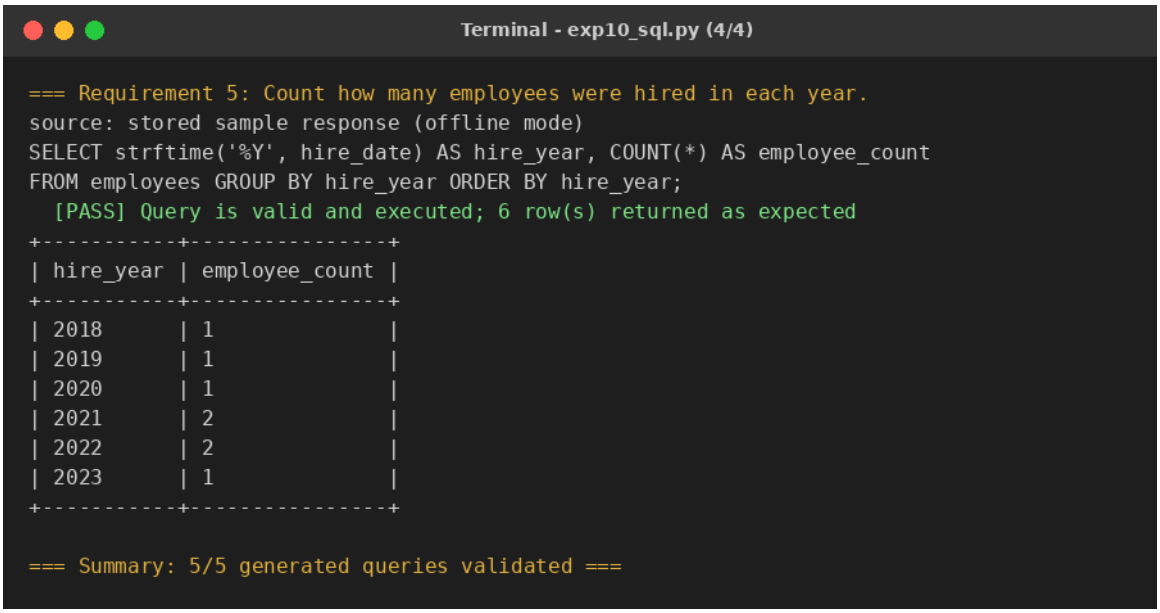


Figure 4: Sample output of the program (part 4 of 4)

RESULT

Five SQL queries were generated from the structured prompts. Each query was a valid single SELECT statement, ran without error on the sample database and returned the expected number of rows (6, 4, 3, 1 and 6), so all 5 of 5 queries were validated. In the run shown, stored sample responses were used in place of a live API call.

CONCLUSION

Giving the LLM the database engine, schema, table details and explicit rules produces accurate SQL that uses the correct tables and columns. Executing the generated query on a test database and checking the results is an effective way to validate it before using it in real applications.